

Grade 8 Interaction & AI

Robots That Sense and Express

20

SESSIONS

1 h

EACH

mBot2 Rover

TANK + EMOJI SCREEN

Blocks → first Python

CODING MODE

★ Final project: Companion Robot

A robot character with custom emojis, three sensor-triggered behaviors, one ML interaction and full controller drive — presented at a Personality Fair with visitor feedback.

Grade 8 at a Glance

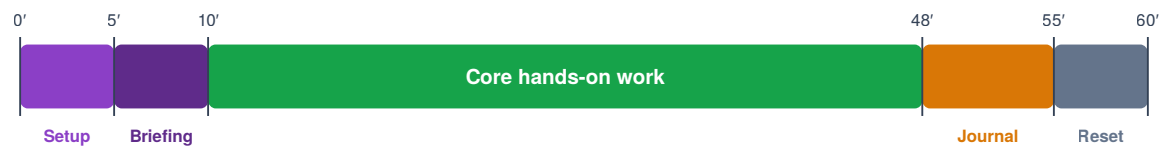
Learning objectives — by the end of the year, students can:

- Rebuild into the Rover tank; explain tracked vs. wheeled trade-offs
- Design custom emojis and animations; build sensor-driven personality
- Train, break and de-bias an image-recognition model
- Use generative AI deliberately; write reusable prompt recipes
- Master the controller; read and edit first Python

The 20 sessions, colored by unit * in session titles = buffer session (tolerates compression if the schedule slips)



Every session runs on the same rhythm



Non-negotiables for 1-hour sessions: robots stay assembled between sessions on a labeled cart shelf · code saved to mBlock cloud accounts every session · battery rotation with a per-team battery manager · extension cards at stations so fast teams never wait.

Known friction points this year: the two Track assemblies mirror each other (classic build error) · the Boards ship covered in paper film — peel while sorting · turn calibration changes on tracks — redo it · ML sessions need webcams + good light · the Bluetooth Controller (Rover box) only responds after the robot is programmed (Session 16 teaches it explicitly).

The Boxes — Official Contents Reference

Both boxes are used this year. Session parts panels tag every piece with its source box.

BOX 1 · “mBot2”

CyberPi ×1	Ultrasonic Sensor 2 ×1	M4×8 mm screw ×6
mBot2 Shield ×1 · battery built in	Quad RGB Sensor ×1	M2.5×12 mm screw ×4
Chassis ×1	Motor cable ×2	Line-following track map ×1
Encoder motor ×2	mBuild cable 10 cm ×2	USB cable ×1
Wheel hub ×2	mBuild cable 20 cm ×1	Screwdriver ×1
Slick tyre ×2	M4×25 mm screw ×6	
Mini wheel ×1	M4×14 mm screw ×6	

Quantities from the official packing list. Bluetooth dongle NOT included — Makeblock advises it for older computers (sold separately).

BOX 2 · “Rover Robotics Add-on Pack”

Servo pack ×2 used	Slide beam 2×4 ≥×1	Nut ≥×30
Bluetooth Controller ×1	Slide beam 2×6 ≥×1	Plastic shock absorber spares — unused in build
Track ×2	Plate 3×8 ≥×1	Metal shock absorber ×4 used
Wheel hub ×4 halves used	Shaft collar ×12 used	Long plastic ring ≥×2
Small wheel hub ×2 used	Flange bearing ≥×4	Short plastic ring ≥×8
D shaft ≥×6	Socket wrench ×1	Rubber band ≥×6
Bracket 90° 3×3 ≥×6	Screw M3×6 mm ×12 used	Board 1–8 (body shells) peel paper film
Bracket 120° 3×3 ≥×3	Screw M4×10 mm ≥×30	Quick Start Guide ×1
U bracket ≥×4	Screw M4×22 mm ≥×14	
T plate ×2 used	Screw M4×30 mm ≥×8	

As printed on the box (no quantities given). Online docs: search “mBot2 Rover” at support.makeblock.com.

Review & Teardown

Goal of this hour: Grade 7 concepts refreshed, and the mBot2 fully disassembled with every part sorted.

YOU NEED — PIECES & MATERIALS



**mBot2,
assembled**
last year's build, to
disassemble
mBot2 box



Screwdriver
mBot2 box



Sorting trays
screws / plates /
motors / electronics
Classroom



**Rover add-on
box**
unbox + inventory
vs. its parts chart
**Rover add-on
box**



**Engineering
journal**
Classroom

TEACH IT — THE CONCEPT

Open with a fast quiz game on Grade 7 (sense—decide—act, the 360° rule, calibration, if/forever, the wobble). Use last year's capstone scoreboard photo as the first slide — continuity matters, and it re-activates knowledge this year builds on.

Then the teardown — and teardown is a skill, not destruction. Rule: a part only leaves the robot if it lands in the correct labeled tray (screws by size, plates, motors, electronics). Reverse-reading the assembly guide teaches the same diagram literacy as building. Handle the CyberPi and sensors like eggs; everything gets reused for the tank.

Watch out

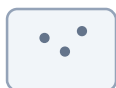
- Photograph full trays — the reference when a screw goes missing next week.
- Motors keep their cables attached; unplugging at the board end saves connectors.

LESSON FLOW (45' CORE)

- 10'** Quiz game in teams — winner picks their cart shelf first.
- 25'** Structured teardown into labeled trays; electronics in their own padded tray.
- 5'** Tray inspection against the parts list — missing parts found NOW, not mid-build.
- 5'** Journal: which Grade 7 skill do you expect to reuse most this year? (Revisit in Session 20.)

● EXPECTED RESULT AT THE END OF THIS SESSION

every part sorted before the tank build starts



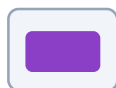
screws M4



plates



motors



electronics

- ✓ Every part in the correct labeled tray, checked against the list
- ✓ Electronics undamaged and stored padded
- ✓ Quiz shows the class can restate all five Grade 7 big ideas

Tank Build I

Goal of this hour: Track assemblies and shock absorbers built — the Rover's chassis takes shape.

YOU NEED — PIECES & MATERIALS



Quick Start Guide

pp. 5–24 — in the Rover box

Rover add-on box



Track x2
x2 — grain direction!

Rover add-on box



Wheel hubs (large + small)
big x4 halves + small x2

Rover add-on box



Shaft collar + flange bearing
flange bearings x4

Rover add-on box



Brackets 90° / 120° / U
T · U · 90° — two mirrored sets

Rover add-on box



Motor cable x2
motors from the mBot2 teardown

mBot2 box



Rover screws + nuts
M4x10 x12 · M4x30 x2 · M4x22 x2 · M2.5x12 x2 + nuts

Rover add-on box



Socket wrench
nut back-off 90° CCW

Rover add-on box



Screwdriver

mBot2 box

TEACH IT — THE CONCEPT

The Rover add-on turns the wheeled mBot2 into a tracked tank. Tracks are a different locomotion philosophy: instead of four small contact points, a long footprint that grips slopes, gravel and grass. The shock absorbers are real springs — the same idea as car suspension, protecting electronics from bumps.

Track tension is today's precision skill: too loose and the track derails on turns; too tight and the motors strain. The test: a track should deflect a few millimeters under a finger press — make each team physically feel a good track before signing off.

LESSON FLOW (45' CORE)

- 5'** Show the finished Rover (photo/video); name the new parts: tracks, hubs, dampers, D shafts.
- 32'** Guide pp. 5–24: build the two mirrored wheel brackets, then the complete left and right track-wheel modules (motor, hubs, bearings, track, outer hub).
- 5'** Free-spin + tension test on both modules — teacher or checker-team fingerpress sign-off.
- 3'** Trays back in order; journal: why must a driven wheel still spin freely on its bearing?

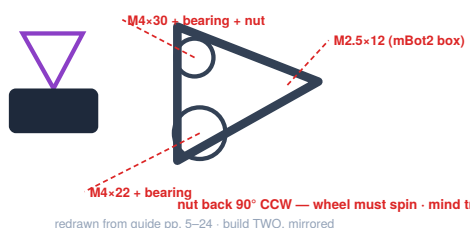
QUICK START GUIDE — THIS SESSION

- Pages** pp. 5–24: brackets x2 → left wheel → right wheel
- Screws** brackets+motor: M4x10 x6 + nuts · per wheel: M4x30 + M4x22 (w/ bearings) + M2.5x12 (mBot2 box)
- Tools** hex end + socket wrench · screw panels print 1:1 — compare

Watch out

- After each hub screw, back the nut 90° counter-clockwise (socket wrench) — the wheel must spin freely or the motors strain.
- Track grain direction matters (magnified inset in the guide); both tracks must point the same way.
- Left and right modules mirror each other; building two identical ones is THE classic error.
- Track links pinch fingers — demonstrate the safe grip first.

EXPECTED RESULT AT THE END OF THIS SESSION



- ✓ Both track-wheel modules complete and mirrored correctly
- ✓ Every hub spins freely after the 90° nut back-off
- ✓ Track tension + grain direction pass the check

Tank Build II + Test Drive

Goal of this hour: The Rover drives — and students can explain why tanks turn differently from cars.

YOU NEED — PIECES & MATERIALS



D shaft
×6 + collars ×12
(M3×6)

Rover add-on
box



Shock absorbers
METAL ×4 — plastic
= spares

Rover add-on
box



Slide beams + plates
2×4 · 2×6 · T · 3×8

Rover add-on
box



Servo pack
×2 — grip arms
(M4×22 ×4)

Rover add-on
box



Boards 1–8
shells 1–8 + bands
— film OFF first

Rover add-on
box



CyberPi
+ shield, charged

mBot2 box



Rover screws + nuts
M4×10/22/30 + nuts
· drivers + socket
wrench

Rover add-on
box



Laptop + mBlock
first tank drive

Classroom

TEACH IT — THE CONCEPT

Finish the build, then first drive. The turning revelation: a tank has no steering wheels — it turns by running the tracks at **different speeds** (skid steering). Left forward + right backward = spin on the spot, something last year's robot could never do.

Structure the comparison: same turn command, last year's video vs. the Rover now. Students articulate the trade-off — tanks turn on a dime and climb, but are slower on smooth floor. Trade-off remains the word of the year.

LESSON FLOW (45' CORE)

- 18'** Guide pp. 25–75 fast lane: chassis (collars, shafts, shocks), head + servos, join halves, shells + bands, arms; cables tucked, QC as in Grade 7.
- 12'** First drive (on the floor, never tables) + spin-on-the-spot test; compare against the Grade 7 turn video.
- 10'** Mini drills: precise 90° and 360° spins — recalibrate turning for tracks (it differs!).
- 5'** Journal: two things tracks do better, two things wheels did better.

QUICK START GUIDE — THIS SESSION

- Pages** pp. 25–75: chassis → head → join → shells → grip arms
- Chassis** pre-assemble 12 collars (M3×6, screw must not enter the hole) · metal shocks ×4 (plastic = spares)
- Head** servos ×2 (M4×22 ×4 + nuts) drive the L1/R1 arms · M4×10/22/30 + socket wrench throughout

BLOCK SCRIPT (REBUILD THIS IN MBLOCK)

```
when button A pressed
  set motors L +40 / R -40 RPM  <- spin
  in place
  wait 1 s
  stop moving
```

Watch out

- The guide **CROSSES** the motor cables (left → right port, right → left); board film off and servo cables routed **BEFORE** the shells close.
- Turn calibration from Grade 7 is now wrong — tracks slip differently; recalibrate before precision work.

EXPECTED RESULT AT THE END OF THIS SESSION



tracks: turns on the spot
(skid steer)



wheels: wider arc
(last year's base)

- ✓ Rover drives and spins on the spot
- ✓ Turning recalibrated for tracks (90° within a few degrees)
- ✓ Team stated the tracks-vs-wheels trade-off in their own words

Terrain Lab

Goal of this hour: A real dataset: Rover speed and climbing ability measured across surfaces.

YOU NEED — PIECES & MATERIALS



Rover,
assembled
QC'd

Rover add-on
box



Laptop + mBlock

Classroom



Ramp board
board on books

Classroom



Protractor
or phone
inclinometer

Classroom



Ruler / tape
measure
2 m speed course

Classroom



Stopwatch
3 trials each

Classroom



Engineering
journal
data tables

Classroom

TEACH IT — THE CONCEPT

Today is experimental method dressed as play. Two experiments: (1) speed over a fixed 2 m distance on different surfaces — tile, carpet, doormat, gravel tray if available; (2) maximum climbable slope using a board propped at increasing angles, measured with a phone inclinometer or a printed protractor.

Insist on fair-test discipline: same robot, same battery level, same command, three trials per condition, record all three (not just the best). The table students fill today is the raw material for Session 5's graphs — sloppy data now means garbage graphs tomorrow, and that's a lesson too.

Watch out

- Battery level is the silent confounder — note it beside each trial.
- On the slope board, add a soft landing zone; failed climbs slide back fast.

LESSON FLOW (45' CORE)

- 8'** Design the fair test together: what stays constant, what varies, how many trials.
- 25'** Stations: speed runs on 3+ surfaces; slope board in 5° increments until the Rover fails.
- 8'** Data check: every cell of every table filled, units written.
- 4'** Predictions in the journal: which surface was fastest, and what will the graph look like?

● EXPECTED RESULT AT THE END OF THIS SESSION



measured climb data — testing the manufacturer's claim

- ✓ Speed table: 3+ surfaces × 3 trials, all recorded with units
- ✓ Maximum slope found to the nearest 5°
- ✓ Team can name the two variables they kept constant

Terrain Science

Goal of this hour: Data becomes graphs, and graphs become a verdict on the manufacturer's 40° claim.

YOU NEED — PIECES & MATERIALS



Engineering journal
yesterday's raw data
Classroom



Laptop + mBlock
or graph paper
Classroom



Printed cards
wall-chart paper
Classroom



Rover, assembled
re-runs for outliers
Rover add-on box

TEACH IT — THE CONCEPT

Turn yesterday's tables into bar charts (speed per surface) and find the class's maximum-slope distribution. The concepts behind the numbers: **friction** (grippy surfaces slow top speed but enable climbing) and **torque** (the twisting force that fights gravity on a slope — more slope needs more torque at less speed).

Then the headline: Makeblock claims the Rover climbs 40°. Does class data agree? Maybe teams got 35°, maybe 42° — either way, discuss why lab conditions differ from marketing conditions (surface, battery, track wear). Students just fact-checked a manufacturer with their own data. That habit — test the claim — is the takeaway of the whole unit.

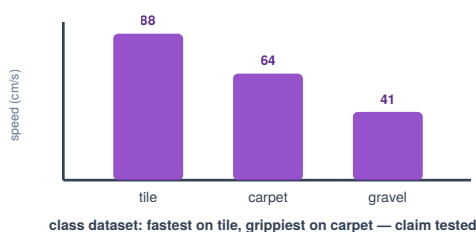
Watch out

- Outlier teams usually had a low battery or a curled mat — hunt the confounder together, don't discard the data silently.
- Keep the wall charts up; they reappear in the capstone as evidence of “our robot's abilities”.

LESSON FLOW (45' CORE)

- 15'** Draw the bar charts (paper or spreadsheet); one chart per team on the wall.
- 10'** Gallery walk: do all teams agree on the fastest surface? Discuss outliers.
- 12'** The 40° verdict: compare class results to the claim; list reasons for gaps.
- 8'** Journal: your verdict sentence — “Our data (supports / partly supports / contradicts) the claim because...”

● EXPECTED RESULT AT THE END OF THIS SESSION



- ✓ Every team produced a labeled bar chart from real data
- ✓ Class reached a written verdict on the 40° claim
- ✓ Students can link friction and torque to one result each

Pixel Art Workshop

Goal of this hour: Each team designs original expressions, pixel by pixel, on the Rover's screen.

YOU NEED — PIECES & MATERIALS



**Rover,
assembled**

Rover add-on
box



Laptop + mBlock
emoji editor

Classroom



Printed cards
printed 16×16 pixel
grids

Classroom



**Engineering
journal**
emoji library list

Classroom

TEACH IT — THE CONCEPT

The Rover's identity lives on its screen. mBlock's emoji editor works like classic pixel art: a 16×16-ish grid, one square at a time. Show how few pixels carry emotion — two dots and a curve read as a face from across the room. That economy is a design superpower.

The constraint challenge: express “confused” in the grid. It's hard — confusion has no iconic emoji shape — which forces real design thinking: tilted eyebrows? A spiral eye? A question mark? Teams draft on paper grids first (fast, erasable), then commit the best to the editor and push it to the actual screen. Seeing your pixels glow on hardware never gets old.

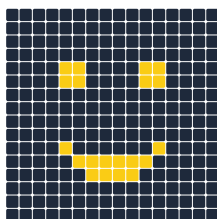
LESSON FLOW (45' CORE)

- 8'** Pixel-economy demo: build a smiley live, removing pixels until it stops reading as a face.
- 12'** Paper drafts: each student sketches “confused” on printed grids; team picks one.
- 20'** Digitize in the emoji editor and display on the Rover; iterate once after seeing it at distance.
- 5'** Screen parade: Rovers line up showing their faces; class guesses each emotion.

Watch out

- Judge emojis from 2 meters away — screens read differently at distance.
- Save every emoji with a clear name (confused_v2); the capstone will reuse this library.

● EXPECTED RESULT AT THE END OF THIS SESSION



a 16×16 pixel expression, drawn frame by frame

- ✓ “Confused” displayed on the Rover and correctly guessed by another team
- ✓ At least two original emojis saved to the team library
- ✓ Paper-draft-first workflow followed

Animation

Goal of this hour: Emojis come alive: multi-frame animations tied to buttons for three mood states.

YOU NEED — PIECES & MATERIALS



Rover,
assembled
Rover add-on
box



Laptop + mBlock
Classroom



Engineering
journal
frame storyboards
Classroom

TEACH IT — THE CONCEPT

Animation = frames + timing, exactly like a flip-book. A blink is three frames (open, half, closed) played fast; a bounce is position shifts. Two principles go a long way: **timing** (sleepy is slow, excited is fast — same frames, different feel) and **anticipation** (a tiny wind-up before a big move reads as alive).

Build three mood states — happy, sleepy, surprised — each an animation loop, each bound to a controller/CyberPi button. This is a state machine without the vocabulary yet: the robot is always in exactly one mood, and buttons switch between them. Session 9 names this idea properly.

LESSON FLOW (45' CORE)

- 8'** Flip-book demo: three-frame blink live; change the frame delay, feel the mood shift.
- 22'** Teams build three mood animations (2–4 frames each) from their emoji library.
- 10'** Wire each mood to a button; test switching cleanly between moods.
- 5'** Mood quiz: teams switch moods on command; class names each one.

BLOCK SCRIPT (REBUILD THIS IN MBLOCK)

```
when button A pressed      <- HAPPY
  forever
    show emoji happy_1 , wait 0.3 s
    show emoji happy_2 , wait 0.3 s

when button B pressed      <- SLEEPY
  forever
    show emoji sleepy_1 , wait 0.8 s
    show emoji sleepy_2 , wait 0.8 s
```

Watch out

- Two mood loops running at once = flickering chaos; each button must stop the previous loop first.
- Slow timing (0.6–0.9 s) reads as calm; under 0.2 s reads as panic — use it deliberately.

● EXPECTED RESULT AT THE END OF THIS SESSION



HAPPY



SLEEPY



SURPRISED

three mood states, each on its own button

- ✓ Three distinct mood animations, each on its own button
- ✓ Switching moods never shows two animations fighting
- ✓ Class correctly named all three moods from the screen alone

Reactive Personality I

Goal of this hour: The Rover reacts emotionally to the physical world: shake, darkness, sound.

YOU NEED — PIECES & MATERIALS



Rover,
assembled
Rover add-on
box



Laptop + mBlock
Classroom



Engineering
journal
threshold notes
Classroom

TEACH IT — THE CONCEPT

Now moods stop being button-driven and become **sensor**-driven — the robot appears to feel its environment. Three mappings using CyberPi's built-in sensors: gyroscope detects a shake → dizzy face; light sensor sees darkness → sleepy; microphone hears a clap → startled jump (a quick backward hop sells it).

Thresholds return from Grade 7, but now they tune *personality*: a low shake threshold makes a skittish robot; a high one makes it stoic. Let teams tune their robot's temperament on purpose — “ours is jumpy, ours is chill” — and suddenly parameters have character. That reframing is the bridge to the capstone.

LESSON FLOW (45' CORE)

- 8'** Read raw sensor values on screen: tilt/shake numbers, light level, loudness. Find natural ranges.
- 22'** Build the three reactions; tune thresholds to taste.
- 10'** Personality demo circle: shake, darken (hands over sensor), clap — robots react in character.
- 5'** Journal: your robot's temperament in one sentence + the thresholds that create it.

BLOCK SCRIPT (REBUILD THIS IN MBLOCK)

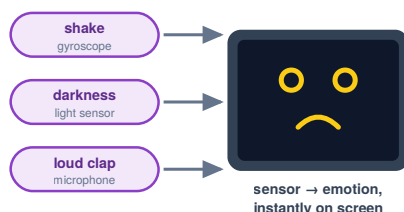
```

forever
  if shake strength > 40
    show emoji dizzy , play sound wobble
  else if light level < 15
    show emoji sleepy
  else if loudness > 70
    show emoji shocked
    move backward 5 cm fast <- the
startle hop
  else
    show emoji neutral
  
```

Watch out

- Classroom clapping escalates — agree the demo protocol before coding the microphone.
- The final else (neutral face) matters: without it the robot freezes in its last emotion.

● EXPECTED RESULT AT THE END OF THIS SESSION



- ✓ All three sensor→emotion mappings work reliably
- ✓ Thresholds tuned deliberately and recorded
- ✓ Robot returns to neutral when nothing happens

Reactive Personality II

Goal of this hour: A mood variable with decay makes emotions persist — plus a structured ethics talk.

YOU NEED — PIECES & MATERIALS



Rover,
assembled

Rover add-on
box



Laptop + mBlock

Classroom



Engineering
journal
ethics position
sentence

Classroom

TEACH IT — THE CONCEPT

Real moods linger. Introduce a **mood variable** (say -5 to $+5$): a clap adds $+3$, a shake -2 , and every few seconds mood drifts one step back toward 0 (decay). The displayed face reads the variable: high = beaming, low = grumpy, middle = neutral. This is a genuine state machine with memory — the robot's reaction now depends on its history, not just the current instant.

Then the 15-minute ethics discussion, structured, not a free-for-all: *Is the robot feeling anything?* Students just built the mechanism, so they know the answer is no — it's a variable. But it *looks* like feeling, and people bond with things that look like feeling (name a Roomba, thank a voice assistant). Introduce the word **anthropomorphism**. Land on the question that matters for the capstone: is designing a machine that appears to feel honest? When is it useful (companion robots for the elderly) and when is it manipulative? No forced conclusion — the discussion continues in their 1-page reflection (assessment).

LESSON FLOW (45' CORE)

- 20'** Build the mood variable: events adjust it, a decay loop drifts it to 0 , faces read it.
- 8'** Cross-team test: interact with another team's robot; can you read its mood history?
- 15'** Structured ethics discussion (think–pair–share): is it feeling? Does it matter that it isn't?
- 2'** Journal: one sentence you'll defend in the ethics reflection.

BLOCK SCRIPT (REBUILD THIS IN MBLOCK)

```
set mood to 0

when loudness > 70: change mood by +3
when shake > 40: change mood by -2

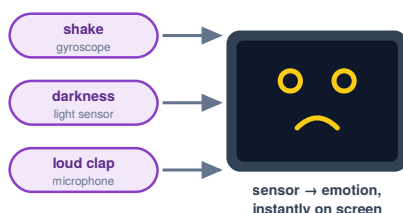
every 3 s:                                <- decay
  if mood > 0: change mood by -1
  if mood < 0: change mood by +1

forever
  if mood >= 3: show emoji beaming
  else if mood <= -3: show emoji grumpy
  else: show emoji neutral
```

Watch out

- Clamp mood to ± 5 or one clapping frenzy pins it forever.
- In the discussion, ban “it's just dumb” as a full answer — require a because.

● EXPECTED RESULT AT THE END OF THIS SESSION



- ✓ Mood persists after the trigger ends and visibly decays back to neutral
- ✓ Another team correctly inferred recent history from the robot's face
- ✓ Every student voiced or wrote one defensible position in the ethics discussion

Train Your Robot

Goal of this hour: A working image classifier, trained by the students, controlling the Rover.

YOU NEED — PIECES & MATERIALS



**Rover,
assembled**

Rover add-on
box



Laptop + mBlock
ML 2.0 access

Classroom



Webcam
1 per team or station

Classroom



**Engineering
journal**
vocabulary check

Classroom

TEACH IT — THE CONCEPT

mBlock's Machine Learning 2.0 lets students train an image classifier in the browser: define classes (thumbs-up / thumbs-down / flat hand), show the webcam ~20 examples of each, and the model learns to distinguish them. No formulas needed — but three words become permanent vocabulary: **training data** (the examples), **classes** (the categories), **confidence** (how sure the model is, in %).

Then wire predictions to behavior: thumbs-up → drive forward; thumbs-down → back up; flat hand → stop. The room goes quiet the first time a robot obeys a gesture — use that moment to ask the grounding question: *did anyone program "thumbs-up" rules?* No — the model learned from examples. That's the difference between programming and training, stated in one sentence.

LESSON FLOW (45' CORE)

- 8'** Create the three classes; discuss what varied examples mean (angles, distances, hands).
- 12'** Collect ~20 samples per class per team; watch live confidence as you train.
- 18'** Connect predictions to Rover actions; test with the trainer, then with a teammate.
- 7'** Vocabulary check: each student uses training data / class / confidence correctly in one sentence.

BLOCK SCRIPT (REBUILD THIS IN MBLOCK)

```
forever
  if recognition result = thumbs_up
    and confidence > 80%
      move forward at 40 RPM
  else if result = thumbs_down
    move backward at 40 RPM
  else if result = flat_hand
    stop moving
```

Watch out

- Webcam + decent lighting per team is the real constraint — plan stations if short on cameras.
- The confidence threshold (>80%) prevents twitchy misfires; let students feel the difference without it.

EXPECTED RESULT AT THE END OF THIS SESSION



- ✓ Model distinguishes all three gestures at high confidence for its trainer
- ✓ Rover obeys gestures through the confidence-gated code
- ✓ All three vocabulary words used correctly by every team

Break the Model

Goal of this hour: Students discover — by sabotage — that a model is only as good as its data.

YOU NEED — PIECES & MATERIALS



Rover,
assembled
Rover add-on
box



Laptop + mBlock
Classroom



Webcam
Classroom



Engineering
journal
sabotage log
Classroom

TEACH IT — THE CONCEPT

Great engineers break things on purpose. Today's lab: make your classifier fail, and document exactly how. Four sabotage stations: (1) retrain with only 3 examples per class — how accurate now? (2) train in bright light, test in dim; (3) train with one person's hand, test with another's; (4) add a "nothing" background class — does it fix false triggers?

Each failure maps to a data-quality rule: **more examples help** • **test conditions must match training conditions** • **data must cover the variety you'll meet** • **models are confidently wrong, not humble** (they output a top class even for nonsense — unless you gave them a nothing-class). These four rules are the entire practical core of data quality, discovered rather than lectured — and rule 3 is the doorway to Session 13's bias discussion.

LESSON FLOW (45' CORE)

- 5'** Brief: your mission is to break it — and write down HOW it broke.
- 28'** Rotate the four sabotage stations; log accuracy impressions at each.
- 8'** Harvest the four rules on the board from the teams' findings.
- 4'** Journal: the rule that surprised you most, with your evidence.

Watch out

- Keep a good backup model saved — sabotage must be reversible for Session 12.
- Station 3 (different hands) is the important one; make sure every team hits it.

● EXPECTED RESULT AT THE END OF THIS SESSION



- ✓ Team broke the model in at least three distinct ways, each logged
- ✓ All four data-quality rules reconstructed by the class
- ✓ Backup good model intact for next session

Emotion Mirror

Goal of this hour: The kit's flagship project: the Rover mirrors your facial expression on its screen.

YOU NEED — PIECES & MATERIALS



**Rover,
assembled**

Rover add-on
box



Laptop + mBlock

Classroom



Webcam
good lighting
matters

Classroom



**Engineering
journal**
per-person accuracy
notes

Classroom

TEACH IT — THE CONCEPT

Everything converges: train classes on facial expressions (smile / frown / neutral — add surprised if strong), and pipe the prediction to the emoji library from Sessions 6–7. Smile at the Rover; it smiles back. Technically it's Session 10's pipeline with faces instead of hands — say so explicitly, because recognizing “same pattern, new data” is what transfer of learning literally is.

Faces are harder than hands: lighting, distance and angle matter more, and the Session 11 rules immediately apply (varied examples! matched conditions!). Teams that struggle should diagnose using the four rules before adding data blindly — the debug discipline meets the data discipline.

LESSON FLOW (45' CORE)

- 10'** Train the expression classes; deliberately varied examples (distance, tilt, glasses on/off).
- 20'** Connect to the emoji library; tune the confidence threshold until mirroring feels instant but stable.
- 10'** Mirror booth: each teammate tests; note for whom it works best (data for Session 13).
- 5'** Journal: which of the four data rules did you use today, and where?

BLOCK SCRIPT (REBUILD THIS IN MBLOCK)

```

forever
  if face result = smile and confidence > 75%
    show emoji happy_big
  else if result = frown
    show emoji sad
  else
    show emoji neutral
  
```

Watch out

- A model trained on one teammate often fails on another — don't fix it yet; that observation IS Session 13.
- Neutral as the fallback class prevents emotional flickering.

EXPECTED RESULT AT THE END OF THIS SESSION



you smile...



...Rover smiles back

- ✓ Rover mirrors at least smile / frown / neutral for its trainer
- ✓ Confidence threshold tuned: responsive but not flickering
- ✓ Per-person accuracy notes recorded for next session

Mirror Extensions + Bias *

Goal of this hour: Expressions trigger movement — and the class confronts why the model works better for some people.

YOU NEED — PIECES & MATERIALS



Rover,
assembled

Rover add-on
box



Laptop + mBlock

Classroom



Webcam

Classroom



Engineering
journal
bias before/after
data

Classroom

TEACH IT — THE CONCEPT

Extend the mirror: smile → happy spin; frown → robot backs away sadly; surprise → startle hop. Emotion now drives *behavior*, which is exactly the capstone's ML requirement — today's build can be lifted straight into it.

Then the conversation the data has been preparing: pool the class's per-person accuracy notes. The pattern will be there — models work best for the people they were trained on, and degrade for faces, skin tones, glasses, or lighting they never saw. Name it: **bias** — not malice, but missing data. Connect to the real world briefly and concretely: commercial face systems have shown exactly this failure at scale. The fix students already know from Session 11: representative data. Let them apply it — retrain with everyone, re-measure, and watch the gap shrink. Bias diagnosed, explained, and reduced with their own hands, in one hour.

LESSON FLOW (45' CORE)

- 15' Build expression → movement reactions on top of the mirror.
- 10' Pool accuracy notes on the board; find the works-best-for-whom pattern.
- 12' Retrain with every teammate's face; re-test; compare before/after.
- 8' Discussion: where would this failure matter in the real world? (Seeds the ethics reflection.)

Watch out

- Buffer session: the movement extension can shrink to one reaction if time is short — the bias block cannot.
- Keep the tone investigative, not accusatory: the model isn't mean, it's under-fed.

● EXPECTED RESULT AT THE END OF THIS SESSION



you smile...



...Rover smiles back

- ✓ At least two expression-triggered movements working
- ✓ Class identified the accuracy gap and explained it as a data problem
- ✓ Retrained model measurably narrowed the gap

AI Emojis

Goal of this hour: Students generate Rover expressions with AI prompts — and learn that wording is engineering.

YOU NEED — PIECES & MATERIALS



Rover,
assembled

Rover add-on
box



Laptop + mBlock
online — generation
needs the network

Classroom



Engineering
journal
save ALL outputs

Classroom

TEACH IT — THE CONCEPT

mBlock's Magic Emoji feature generates screen expressions from choices/prompts: emotion, color, style. It's generative AI in miniature — and small is perfect for a first controlled experiment: hold the emotion fixed ("excited"), vary only the style wording across three attempts, and compare outputs side by side.

The findings to surface: the same prompt can give different results (generative AI is not a vending machine); specific prompts beat vague ones ("excited, wide sparkling eyes, orange, cartoon" vs. "happy"); and word order/emphasis matters. Contrast with Session 10: a *classifier* answers "which class is this?", a *generator* creates something new. Two different animals, both called AI.

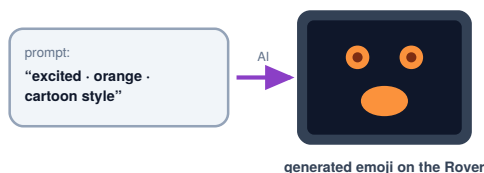
Watch out

- Generation needs connectivity — verify accounts and network before class.
- Require saving ALL outputs, including failures; the comparison is the data.

LESSON FLOW (45' CORE)

- 8'** Demo one generation live; run the same prompt twice — note the outputs differ.
- 22'** Controlled experiment: fixed emotion, three style-wordings, outputs saved side by side.
- 10'** Gallery: teams show their triptychs; class extracts "what made prompts work" rules.
- 5'** Journal: your best prompt, verbatim, and why you think it worked.

● EXPECTED RESULT AT THE END OF THIS SESSION



- ✓ Three-output controlled comparison saved and shown
- ✓ Team articulated two concrete prompt-improvement rules
- ✓ Students can state the classifier-vs-generator distinction

Prompt Recipes

Goal of this hour: A reusable prompt recipe card per student, and a hybrid AI + hand-edited signature emoji.

YOU NEED — PIECES & MATERIALS



Rover,
assembled

Rover add-on
box



Laptop + mBlock

Classroom



Printed cards
recipe-card template

Classroom



Engineering
journal
recipe card is
graded

Classroom

TEACH IT — THE CONCEPT

Formalize yesterday's findings into a personal **prompt recipe card**: a fill-in template — [emotion] + [2–3 visual details] + [color] + [style] — with the student's own tested examples. Writing the recipe forces the generalization; keeping the card makes prompt skill portable to every AI tool they'll ever meet. That's the quiet payload of this session.

Then the hybrid workflow, which mirrors real creative AI practice: generate a base with the recipe, then open it in the pixel editor and hand-fix what the AI got wrong (eyes too small, mouth off-center). AI for the draft, human for the finish. The result becomes the team's **signature emoji** — the face of their capstone character.

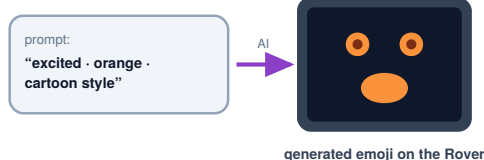
Watch out

- Peer-testing recipes is the quality check: a good recipe works in someone else's hands.
- Timebox the pixel-polishing — perfectionists will happily spend the hour on one eyebrow.

LESSON FLOW (45' CORE)

- 12'** Write and peer-test recipe cards: swap cards, run a stranger's recipe, compare intent vs. output.
- 22'** Hybrid build: generate with the recipe → hand-edit pixels → signature emoji on the Rover.
- 8'** Signature parade + one-line "design statement" per team.
- 3'** File the recipe card in the journal — it's a graded artifact.

● EXPECTED RESULT AT THE END OF THIS SESSION



- ✓ Recipe card produced and successfully run by another student
- ✓ Signature emoji = AI base + visible hand edits, displayed on the Rover
- ✓ Design statement links the emoji to the intended character

Controller Mastery *

Goal of this hour: Full manual control: joysticks drive, buttons trigger character — tested on an obstacle course.

YOU NEED — PIECES & MATERIALS



**Rover,
assembled**
Rover add-on
box



Laptop + mBlock
program the listener
FIRST
Classroom



**Bluetooth
Controller**
in the Rover box
Rover add-on
box



**Makeblock BT
dongle**
advised — sold
separately
Sold separately



Boxes / obstacles
slalom + gate
Classroom



Masking tape
course marks
Classroom



Stopwatch
gauntlet times
Classroom

TEACH IT — THE CONCEPT

The Bluetooth controller (2 joysticks, 15 buttons) turns the Rover into a drivable character. Critical gotcha, stated up front and loudly: the controller does nothing until the robot runs a program that *listens* for it — the robot must be programmed first. This confuses every first-timer; teaching it explicitly saves twenty minutes of “it’s broken”.

Mapping is design: left stick = drive (tank-style: stick Y values to each track), buttons = moods, sounds, speed modes (turbo/precise). Have teams write their mapping on a paper “controller card” — pilots do this. Then the driving gauntlet: a cone/box slalom, a narrow gate, a reverse park. Precise manual driving builds the motor intuition that makes autonomous tuning meaningful.

LESSON FLOW (45' CORE)

- 5'** The gotcha demo: dead controller → load listener program → alive. Never forgotten.
- 18'** Build the mapping: sticks drive, buttons for moods/sounds/speed modes — and L1/R1 for the grip arms, matching Makeblock’s official demo program; fill the controller card.
- 17'** Obstacle gauntlet time trials: slalom, gate, reverse park — penalties for touches.
- 5'** Journal: your mapping card + which speed mode won the gauntlet and why.

BLOCK SCRIPT (REBUILD THIS IN MBLOCK)

```

forever
  set motor L to (left stick Y) RPM
  set motor R to (right stick Y) RPM

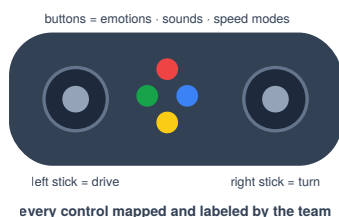
when controller button Δ pressed
  show emoji happy , play sound yay

when controller button R1 pressed
  set speed_mode to turbo
  
```

Watch out

- Buffer session: the gauntlet can shrink to the slalom only.
- Two sticks (one per track) is harder but more “tank-real” than one-stick arcade drive — offer both, let teams choose.
- The official example program maps L1/L2/R1/R2 to the grip arms — a great reference for teams to study.

● EXPECTED RESULT AT THE END OF THIS SESSION



- ✓ Controller drives the Rover with the team’s documented mapping
- ✓ At least four character buttons live (moods / sounds / speed modes)
- ✓ Gauntlet completed; time and penalties recorded

Blocks → Python

Goal of this hour: Students read the Python behind their blocks and successfully predict a code change.

YOU NEED — PIECES & MATERIALS



**Rover,
assembled**

Rover add-on
box



Laptop + mBlock

Python view

Classroom



USB cable
stable connection
helps

mBot2 box



**Engineering
journal**

3 translations

Classroom

TEACH IT — THE CONCEPT

mBlock has a Python view: the blocks students know, as text. Put a familiar program (the loop-square) side by side with its Python and let them decode it: `for i in range(4)` is repeat-4, indentation is the “inside” of the block, `mbot2.straight(30)` is move-forward-30. They already know what this program does — so they’re not learning to program, just learning a new notation for programs they own.

The method is predict-then-run: change one value in the text (`30` → `50`, `range(4)` → `range(6)`), write down the prediction, run, verify. End with the mini-challenge: hand-write the pentagram (5 sides, 144°) in Python from memory of the block version. Grade 9 opens exactly here — this session is its front porch.

LESSON FLOW (45' CORE)

- 10'** Side-by-side decoding of the loop-square; class maps each block to its line.
- 15'** Predict-then-run: three single-value edits in Python, predictions written first.
- 15'** Mini-challenge: pentagram in Python; run it on the Rover.
- 5'** Journal: three block→Python translations you’re sure of.

PYTHON (MBLOCK → PYTHON MODE)

```
import cyberpi
import mbot2

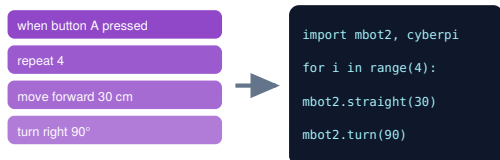
# the Session-8 square, as text
for i in range(4):
    mbot2.straight(30) # cm
    mbot2.turn(90)    # degrees

cyberpi.audio.play('yay')
# challenge: make this the pentagram
```

Watch out

- Function names can differ slightly by mBlock version — have students trust the editor’s autocomplete over any handout.
- Indentation errors are the #1 stumble; show one on purpose and read the error message together.

● EXPECTED RESULT AT THE END OF THIS SESSION



the same program, in blocks and in Python

- ✓ Team correctly predicted all three edits before running
- ✓ Pentagram runs from hand-written Python
- ✓ Students can explain what indentation means in Python

Capstone: Character Design

Goal of this hour: A complete character storyboard, approved before any code is written.

YOU NEED — PIECES & MATERIALS



Engineering journal
character sheet — deliverable
Classroom



Printed cards
sheet template + rubric
Classroom



Rover, assembled
reference only — no coding today
Rover add-on box

TEACH IT — THE CONCEPT

The brief, in full: design a robot **character** with (a) a name and visual identity built on the signature emoji, (b) at least three sensor-triggered behaviors, (c) one ML-powered interaction, (d) full controller drive mode. Presented at the Personality Fair (Session 20) where real visitors interact and leave feedback.

Design before code — the Grade 7 planning gate returns, with a character-sheet template: name, one-line personality, emoji set, behavior table (trigger → reaction → what it expresses), ML interaction, controller card. The best characters are *coherent*: a shy robot startles big and approaches slowly; a party robot celebrates everything. Coherence is a rubric line — say so. Sign-off question at the gate: “why does THIS robot react THIS way?” If the answer is character-driven, they’re ready.

Watch out

- Everything on the sheet must already exist in Sessions 6–17 — the capstone assembles, it doesn’t invent.
- Steer ML choices to the mirror (S12) or gestures (S10) — both are proven pipelines.

LESSON FLOW (45’ CORE)

- 8’** Brief + rubric + tour the fair format so the audience is real from the start.
- 12’** Character sketching: name, personality line, mood board of emojis.
- 18’** Fill the behavior table + ML choice + controller card; optional backpack decoration plan.
- 7’** Sign-off interviews with the coherence question.

● EXPECTED RESULT AT THE END OF THIS SESSION



- ✓ custom emoji set (5 moods)
- ✓ shake → dizzy spin
- ✓ clap → happy dance
- ✓ ML: waves back at you
- ✓ full controller drive

- ✓ Character sheet complete: identity + 3 behaviors + ML + controls
- ✓ Every behavior traces to a session where it was built
- ✓ Sign-off passed on the coherence question

Capstone: Build *

Goal of this hour: The full character runs end-to-end, rehearsed for visitor interaction.

YOU NEED — PIECES & MATERIALS



Rover,
assembled

Rover add-on
box



Laptop + mBlock

Classroom



Bluetooth
Controller

Rover add-on
box



Webcam
if ML = mirror

Classroom



Printed cards
interaction sign

Classroom



mBot2 Shield
charge overnight!

mBot2 box

TEACH IT — THE CONCEPT

Implementation sprint, in flowchart order: identity first (emoji set + idle animation — quick win), then the three sensor behaviors, then the ML interaction, controller mode last. One piece verified at a time; the mood-loop conflicts from Session 7 will resurface here — the fix is the same (each state cleanly stops the previous one).

Rehearse for strangers: a visitor won't know to clap twice at 30 cm. Teams write a small "interaction sign" ("Clap! • Shake gently! • Smile at me!") and test their robot with a member of another team playing naive visitor. What confuses a classmate will confuse a fair visitor twice as much.

Watch out

- Buffer session: the coherence and the three behaviors are the core; ML can fall back to the proven S12 mirror unchanged.
- Freeze code at the end of the hour — fair-morning tinkering is the classic self-destruct.

LESSON FLOW (45' CORE)

- 30'** Build in order: identity → behaviors → ML → controller; verify each before the next.
- 10'** Naive-visitor test with another team; fix the top confusion found.
- 5'** Write the interaction sign + station checklist for the fair (battery! calibration!).

● EXPECTED RESULT AT THE END OF THIS SESSION



"ZAPPY"

- ✓ custom emoji set (5 moods)
- ✓ shake → dizzy spin
- ✓ clap → happy dance
- ✓ ML: waves back at you
- ✓ full controller drive

- ✓ All character-sheet features run in one continuous demo
- ✓ A naive visitor triggered every behavior using only the sign
- ✓ Code frozen; fair checklist written

Personality Fair

Goal of this hour: Visitors meet the robots; teams present; feedback is collected and reflected on.

YOU NEED — PIECES & MATERIALS



Rover,
assembled
code frozen

Rover add-on
box



Bluetooth
Controller

Rover add-on
box



Webcam

Classroom



Printed cards
visitor feedback
cards

Classroom



mBot2 Shield
fully charged

mBot2 box



Masking tape
station setup

Classroom

TEACH IT — THE CONCEPT

Run the fair as stations: each team's Rover on a table with its interaction sign and feedback cards ("Which emotion did you see? ★ rating • What surprised you?"). Invite an audience if at all possible — another class, teachers, younger students; robots performing for strangers is a different sport, and students rise to it.

Close with the year's real question: *what made an interaction feel alive?* Harvest answers — usually: it responded fast, it looked at me, it had moods, it remembered (the decay variable!). Note that every answer is a design decision they made. Then the Grade 9 teaser: "next year, your robot gets an arm and a job." The ethics reflection (1 page, assessment) goes home with them: anthropomorphism, training data, bias — they have lived all three.

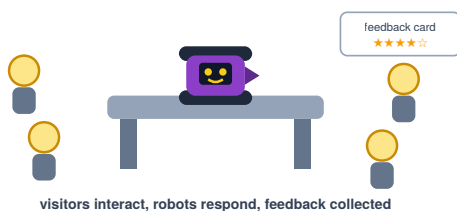
LESSON FLOW (45' CORE)

- 8'** Station setup: robots, signs, feedback cards, fresh batteries.
- 25'** The fair: visitors rotate; half of each team hosts while the other half visits, then swap.
- 8'** Read feedback cards; each team shares its favorite and its most useful criticism.
- 4'** "Alive" harvest + Grade 9 teaser + ethics reflection handed out.

Watch out

- Charge everything beforehand; a dead robot at a fair is heartbreak.
- Photograph each station — these images open Grade 9, Session 1.

● EXPECTED RESULT AT THE END OF THIS SESSION



- ✓ Every robot was operated successfully by at least three visitors
- ✓ Feedback cards collected and one improvement named per team
- ✓ Ethics reflection assigned with lived examples to draw on